

WEEK5 Constraints and Triggers

Constraints & Triggers — Enforcing Data Integrity and Automating Reactions to Change

Study notes combining six Stanford lecture videos (motivation, constraints demo, referential integrity, triggers intro, triggers demo parts 1 & 2) and the CSC404 *Lecture 9* slides. The topic splits cleanly in two:

Part A — Constraints (static rules the data must always satisfy) and **Part B — Triggers** (dynamic rules that react to changes).

0. The Big Picture

Concept	Nature	What it does
Constraints	Static	Define which database states are <i>allowable</i> . Checked on every change; violations are rejected (or auto-repaired in one specific case).
Triggers	Dynamic	Watch the database for events (inserts/updates/deletes), test a condition, and automatically run an action. Can themselves modify the database.

A constraint says *"the database must look like this."* A trigger says *"when this happens, do that."*

Both are part of the **SQL standard**, but real database systems **vary considerably** in what they implement. The SQL standard is the reference; expect each DBMS (PostgreSQL, SQLite, MySQL, ...) to deviate.

PART A — CONSTRAINTS

1. What Are Integrity Constraints?

An **integrity constraint** (also called just a **constraint**) is an **assertion that data in the database must always satisfy**.

Constraints go **beyond** the structural and type restrictions you get for free from the schema (e.g., "GPA is a FLOAT"). They are **semantic** — they encode rules from the *application*, not just from the data types.

Examples (from the college-admissions database)

Constraint (in English)	Type
GPA > 0.0 AND GPA ≤ 4.0	Range check on one column
enrollment < 75000	Range check on one column
decision IN ('Y','N',NULL)	Allowed-values check
If major = 'CS' then decision IS NULL (no CS decisions made yet)	Logical implication
If sizeHS < 200 then student isn't admitted to colleges with enrollment > 30000	Cross-table implication

The last one is contrived but shows the **expressive power** the SQL standard allows.

2. Why Use Constraints?

Several distinct reasons:

Reason	What it catches
Catching data-entry errors	Typos at INSERT time (e.g. GPA = 39.0 instead of 3.9).
Correctness criteria for updates	Bad updates that would push values out of valid range.
Enforcing consistency	Redundant or dependent data stays consistent across the

Reason	What it catches
Telling the system about the data	database. KEY / UNIQUE constraints let the optimizer choose better plans and pick better storage.

3. Types of Constraints — A Taxonomy

Roughly from simplest to most complex:

Type	What it constrains	SQL syntax	Widely supported?
NOT NULL	One attribute may not be NULL	col TYPE NOT NULL	Everywhere
PRIMARY KEY / UNIQUE	Column(s) must have distinct values	PRIMARY KEY (col), UNIQUE (col)	Everywhere
Referential integrity / FOREIGN KEY	A value must exist in another table's referenced column	FOREIGN KEY (col) REFERENCES T(col)	Everywhere
Attribute-based CHECK	Condition on one attribute of a tuple	col TYPE CHECK (cond)	PG, SQLite; MySQL (silently ignored)
Tuple-based CHECK	Condition across attributes of the same tuple	CHECK (cond) at end of CREATE TABLE	PG, SQLite; MySQL
General assertion	Condition spanning multiple tables	CREATE ASSERTION name CHECK (cond)	Not implemented anywhere

PRIMARY KEY implies both UNIQUE and NOT NULL.

4. When Are Constraints Checked?

4.1 Declaration time

There are two moments a constraint can be declared:

When declared	What happens
With the original schema (in CREATE TABLE)	Checked after bulk-loading the initial data.
After the database is already in operation (ALTER TABLE ...)	Checked on the current state at the moment of declaration.

In either case, if existing data violates the constraint, an error is raised and the constraint is rejected.

4.2 Per-modification checking

Once active, the constraint is **re-checked on every modification** that could possibly violate it (a smart system skips changes that are irrelevant — e.g. don't re-check a GPA constraint after an enrollment update).

If a modification would violate, the system **raises an error and undoes the modification**.

4.3 Deferred constraint checking

Sometimes a sequence of modifications passes through invalid intermediate states but ends in a valid one. **Deferred checking** lets you batch:

Mode	Check happens...
IMMEDIATE (default)	After each statement
DEFERRED	At end of transaction (commit time)

A transaction is a group of statements executed as a unit. We'll cover transactions in their own lecture.

5. NOT NULL Constraint

The simplest constraint — an attribute cannot be NULL.

```
CREATE TABLE Student (
  sID INT,
  sName TEXT,
  GPA FLOAT NOT NULL,
  sizeHS INT
);
```

Trace through some inserts

Assume the table above:

Statement	Outcome
INSERT (123, 'Amy', 3.9, 1000)	all values present
INSERT (234, 'Bob', 3.6, NULL)	sizeHS is allowed to be NULL
INSERT (345, 'Craig', NULL, 500)	violates GPA NOT NULL

Updates — there's a subtlety

```
UPDATE Student SET GPA = NULL WHERE sID = 123; -- error: 123 exists
UPDATE Student SET GPA = NULL WHERE sID = 456; -- no error: no rows match
```

Important: A constraint violation requires an *actual attempted change to a row*. If WHERE matches no rows, nothing is updated and the constraint is never tested — **no error**.

6. Key Constraints — PRIMARY KEY and UNIQUE

A **key constraint** declares that the value(s) in a column (or combination of columns) **must be unique across all tuples** in the table.

6.1 PRIMARY KEY

```
CREATE TABLE Student (
  sID INT PRIMARY KEY,
  sName TEXT,
  GPA FLOAT,
  sizeHS INT
);
```

- Each table is allowed **at most one** PRIMARY KEY (hence "primary").
- The table is often physically organised around this key, making PK lookups fast.

- PRIMARY KEY $\Rightarrow$ UNIQUE $\Rightarrow$ NOT NULL (usually — see §6.4).

6.2 UNIQUE — secondary keys

A table can have **any number** of UNIQUE constraints.

```
CREATE TABLE Student (
  sID INT PRIMARY KEY,
  sName TEXT UNIQUE,    -- no two students may share a name
  ...
);
```

6.3 Composite (multi-attribute) keys

A key can span several columns — the **combination** must be unique. Syntax goes **after** the column list:

```
CREATE TABLE College (
  cName TEXT,
  state TEXT,
  enrollment INT,
  PRIMARY KEY (cName, state) -- e.g. "Mason, CA" and "Mason, NY" both allowed
);
```

A real example for Apply:

```
CREATE TABLE Apply (
  sID INT,
  cName TEXT,
  major TEXT,
  decision CHAR(1),
  UNIQUE (sID, cName),    -- each student applies to each college only once
  UNIQUE (sID, major)    -- each student applies to each major only once
);
```

6.4 NULLs and key constraints (a surprise)

The SQL standard and **most systems allow repeated NULLs in UNIQUE columns** — NULLs don't count as "equal" for uniqueness purposes. PRIMARY KEY columns, however, **forbid NULLs entirely** in most systems.

6.5 The "tricky update" — order of evaluation matters

Suppose sID is PRIMARY KEY, with Amy = 123, Bob = 234.

```
UPDATE Student SET sID = sID - 111;    -- which order does the DB pick?
```

- If the system updates **Amy first** (123 → 12), then **Bob** (234 → 123) — no conflict.
- If the system updates **Bob first** (234 → 123), now there's a clash with Amy.

Whether the constraint is violated depends on **internal execution order** — different runs can give different results. This is why **deferred checking** exists: defer the key check until the end and the whole UPDATE is fine.

7. Referential Integrity — Foreign Keys

This is the **most commonly used constraint type in practice**. It deserves its own section.

7.1 The core idea

References between tables are made by **values**, not pointers. Referential integrity says: *every reference points to something real*. It's the database analogue of "no dangling pointers."

Formally: a referential-integrity constraint from **R.A** to **S.B** says

Every non-NULL value in R.A must appear in S.B.

- R.A is called the **foreign key**.
- S.B is the **referenced** attribute. It must be PRIMARY KEY or UNIQUE in S (mostly for efficient implementation, but required by the standard).

7.2 In our college database

Foreign key	References	Meaning
Apply.sID	Student.sID	Every applicant must be a real student
Apply.cName	College.cName	Every applied-to college must exist

Referential integrity is directional. Apply.sID → Student.sID says *every application has a real student*. The reverse direction would say *every student must have applied somewhere* — usually **not** what we want.

7.3 Sample violations

Bad state	Why it violates
Apply (555, 'Stanford', CS, Y) but no student 555 in Student	Apply.sID doesn't reference a real student
Apply (123, 'Yale', CS, Y) but no Yale in College	Apply.cName doesn't reference a real college

7.4 What operations can violate referential integrity?

Operation on...	Operation type	Can it cause violation?
Referencing table (R = Apply)	INSERT	— might insert a value with no match
	UPDATE A	— might change the FK to something invalid
Referenced table (S = Student/College)	DELETE	— orphans the referencing rows
	UPDATE B	— same; now the referencing rows point to a stale value

7.5 What happens when an operation on s would orphan rows in R?

This is the special case where SQL doesn't just say "error" — you choose what should happen:

Option	On DELETE from S	On UPDATE of S.B
RESTRICT (default)	Block the delete; raise error	Block the update; raise error
SET NULL	Set referencing column(s) in R to NULL	Same
SET DEFAULT	Set referencing column(s) to default value	Same
CASCADE	Delete the referencing rows in	Propagate the new value to R's

Option	On DELETE from S	On UPDATE of S.B
	R too	referencing rows

Why "cascade"? Cascading deletes/updates can chain: deleting from College deletes from Apply which might delete from yet another table, and so on. Be careful — a single delete can demolish a large portion of your database!

7.6 SQL syntax

```
CREATE TABLE Apply (
  sID INT,
  cName TEXT,
  major TEXT,
  decision CHAR(1),

  FOREIGN KEY (sID) REFERENCES Student(sID)
    ON DELETE SET NULL,      -- if student deleted, blank out FK
  FOREIGN KEY (cName) REFERENCES College(cName)
    ON UPDATE CASCADE       -- if college renamed, rename everywhere
);
```

Options not specified default to RESTRICT. So in the example above, ON UPDATE for sID and ON DELETE for cName both default to RESTRICT.

7.7 Multi-attribute foreign keys

If the referenced key is multi-column, the foreign key must match column-for-column:

```
CREATE TABLE College (
  cName TEXT,
  state TEXT,
  enrollment INT,
  PRIMARY KEY (cName, state)
);
```

```
CREATE TABLE Apply (
  sID INT,
  cName TEXT,
  state TEXT,
  major TEXT,
  decision CHAR(1),
```

```
FOREIGN KEY (cName, state) REFERENCES College(cName, state)
);
```

7.8 Intra-table referential integrity

A table can have a foreign key referencing **itself** — useful for tree-like data (employees → managers, comments → parent comments).

7.9 Dropping tables

Referential integrity also blocks DROP TABLE. You can't drop Student while Apply still references it — drop the referencing table first, or use CASCADE.

8. CHECK Constraints

8.1 Attribute-based check

Attached to a single attribute, checked on every insert/update **of that row**:

```
CREATE TABLE Student (
  sID INT PRIMARY KEY,
  sName TEXT,
  GPA FLOAT CHECK (GPA > 0.0 AND GPA <= 4.0),
  sizeHS INT CHECK (sizeHS < 5000)
);
```

Inserted row	Outcome
(123, 'Amy', 3.9, 1000)	
(234, 'Bob', 4.5, 1500)	GPA out of range
UPDATE Student SET sizeHS = sizeHS * 6	many rows would exceed 5000

8.2 Tuple-based check

Placed at the end of CREATE TABLE; can reference any attributes of the tuple:

```
CREATE TABLE Apply (
  sID INT,
  cName TEXT,
  major TEXT,
  decision CHAR(1),
```

```
CHECK (decision = 'N' OR cName <> 'Stanford' OR major <> 'CS')
);
```

What does that condition mean? Apply De Morgan: it's logically equivalent to *NOT (decision = 'Y' AND cName = 'Stanford' AND major = 'CS')* — i.e., **nobody is admitted to Stanford CS**. A pure rule expressed in disjunctive form.

8.3 MySQL silently ignores CHECK

At the time of these lectures, **MySQL accepts CHECK syntactically but does not enforce it**. Use PostgreSQL or SQLite for CHECK constraints.

8.4 CHECK with subqueries — the limitation

The SQL standard allows CHECK to contain subqueries — in principle you could implement a foreign key or even a key with a subquery CHECK. **In practice, no major DBMS supports subqueries inside CHECK.**

Example of what the standard *would* allow (but nobody actually does):

```
-- "Every applicant's sID must be a real student" via CHECK:
CREATE TABLE Apply (
  sID INT CHECK (sID IN (SELECT sID FROM Student)),
  ...
);
```

Deceptive even if it worked: a subquery CHECK only re-fires on changes to Apply. If someone later *deletes* from Student, the Apply.sID CHECK is **not** re-evaluated. This is exactly why we have proper referential integrity — it monitors both sides.

9. General Assertions (SQL standard, not implemented)

The most powerful constraint form — conditions spanning **the whole database**, not tied to any single table:

```
CREATE ASSERTION key_assertion CHECK (
  (SELECT COUNT(DISTINCT A) FROM T) = (SELECT COUNT(*) FROM T)
);
```

(A roundabout way of saying "A is a key in T.")

A useful-looking one:

```
CREATE ASSERTION avgGPA CHECK (  
  (SELECT AVG(GPA) FROM Student  
   WHERE sID IN (SELECT sID FROM Apply WHERE decision = 'Y'))  
  > 3.0  
);
```

No SQL system currently implements general assertions. They're in the standard but only theoretical for now — included here because the lecturer covers them. To enforce something like this in a real system, use a **trigger** (Part B).

PART B — TRIGGERS

10. What Is a Trigger?

A **trigger** is a stored **Event–Condition–Action (ECA) rule**:

WHEN *event* occurs **AND** *condition* is true **THEN** perform *action*.

- **Event** — an INSERT, UPDATE, or DELETE on a specific table.
- **Condition** — optional WHEN clause; a boolean tested at trigger time.
- **Action** — one or more SQL statements (or stored procedure calls).

Triggers are stored in the database, run **automatically** by the DBMS, and are part of the schema.

11. Why Use Triggers?

Two main motivations:

Reason	Idea
Move monitoring logic into the database	Instead of every application reimplementing "when X happens, also do Y," put it once in a trigger — more modular, applies uniformly.
Enforce constraints (especially complex ones)	Real DBMSs implement only a subset of SQL constraints. Triggers cover the rest. They can

Reason	Idea
	also automatically repair violations, where ordinary constraints would just error out.

12. Trigger Syntax (SQL Standard)

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF} {INSERT | UPDATE [OF col,...] | DELETE} ON Table
[REFERENCING
  OLD ROW AS o NEW ROW AS n
  OLD TABLE AS ot NEW TABLE AS nt]
[FOR EACH ROW]
[WHEN (condition)]
action;
```

12.1 Timing

Timing	Fires...	Used for
BEFORE	Before the data change	Validate or modify the incoming row
AFTER	After the data change	React to the change (most common)
INSTEAD OF	Replaces the operation	Updating views (covered in the views lecture)

12.2 Granularity — row-level vs statement-level

Granularity	Specified by	When it fires
Row-level	FOR EACH ROW present	Once per affected row
Statement-level	FOR EACH ROW absent	Once per SQL statement , regardless of rows affected

Example: DELETE FROM Student WHERE GPA < 2.0 deletes 10 rows. A row-level trigger fires 10 times; a statement-level trigger fires once.

12.3 Referencing variables — accessing the changed data

Triggering event	Available references
INSERT	NEW ROW, NEW TABLE (inserted data)
DELETE	OLD ROW, OLD TABLE (deleted data)
UPDATE	OLD ROW, NEW ROW, OLD TABLE, NEW TABLE

Granularity	Available
FOR EACH ROW	All four kinds (row + table)
Statement-level	Table-level only (no OLD ROW/NEW ROW)

Confusion alert: OLD TABLE does **not** mean "the table as it was before." It means *the set of tuples that were just modified/deleted*. Likewise NEW TABLE = the set of inserted/updated tuples.

13. DBMS Variations — a Compatibility Cheat Sheet

Feature	SQL standard	PostgreSQL	SQLite	MySQL
Row-level triggers				
Statement-level triggers				
Activation timing	End of statement	End of statement	Immediately after each row	Immediately after each row
OLD ROW / NEW ROW			(auto-bound as old, new)	
OLD TABLE / NEW TABLE				

Feature	SQL standard	PostgreSQL	SQLite	MySQL
Multiple triggers per event				(only one)
CHECK constraints				(ignored)

The lecture demos use **SQLite** because Postgres syntax is awkward; just remember SQLite's two big deviations from the standard:

1. **Row-level only** (no statement-level).
 2. **Immediate activation** after each row, not end of statement.
-

14. A Simple Worked Example — Cascade Delete via Trigger

Without a foreign key, we can still implement cascade-delete behaviour using a trigger. Suppose R.A references S.B:

```
CREATE TRIGGER cascade_delete
AFTER DELETE ON S
REFERENCING OLD ROW AS o
FOR EACH ROW
-- no WHEN clause: always fire
DELETE FROM R WHERE A = o.B;
```

For each row deleted from S, delete every R row whose A matches that row's B.

The **statement-level** equivalent uses OLD TABLE:

```
CREATE TRIGGER cascade_delete_stmt
AFTER DELETE ON S
REFERENCING OLD TABLE AS ot
-- no FOR EACH ROW
DELETE FROM R WHERE A IN (SELECT B FROM ot);
```

The statement-level version is often more efficient — it fires once and handles everything in a single DELETE.

15. Practical Trigger Examples (from the demos)

These use the college-admissions database.

15.1 Auto-apply trigger

"When a new student is inserted, if their GPA is between 3.3 and 3.6, automatically apply them to Stanford (geology) and MIT (biology)."

```
CREATE TRIGGER auto_apply
AFTER INSERT ON Student
FOR EACH ROW
WHEN (new.GPA > 3.3 AND new.GPA <= 3.6)
BEGIN
    INSERT INTO Apply VALUES (new.sID, 'Stanford', 'geology', NULL);
    INSERT INTO Apply VALUES (new.sID, 'MIT', 'biology', NULL);
END;
```

Inserting (111, 'Kevin', 3.5, 1000) causes both Apply rows to appear automatically. Inserting (222, 'Laurie', 3.9, ...) doesn't — her GPA fails the WHEN clause.

15.2 Cascade delete (intra-relation)

"When a student is deleted, also delete their applications." (Mimics referential integrity with ON DELETE CASCADE.)

```
CREATE TRIGGER apply_cascade_delete
AFTER DELETE ON Student
FOR EACH ROW
BEGIN
    DELETE FROM Apply WHERE sID = old.sID;
END;
```

15.3 Cascade update

```
CREATE TRIGGER cName_cascade_update
AFTER UPDATE OF cName ON College
FOR EACH ROW
BEGIN
    UPDATE Apply SET cName = new.cName WHERE cName = old.cName;
END;
```

Renaming Stanford to *"The Farm"* in College propagates to all referencing Apply rows.

15.4 Simulating uniqueness with a BEFORE trigger

```
CREATE TRIGGER college_unique_insert
BEFORE INSERT ON College
FOR EACH ROW
WHEN (EXISTS (SELECT * FROM College WHERE cName = new.cName))
BEGIN
    SELECT RAISE(IGNORE); -- abort the insert
END;
```

SQLite's RAISE(IGNORE) aborts the current statement silently.
RAISE(ABORT, 'message') raises an error.

15.5 "Cap" trigger — close a college at 10 applications

```
CREATE TRIGGER cap_apps
AFTER INSERT ON Apply
FOR EACH ROW
WHEN ((SELECT COUNT(*) FROM Apply WHERE cName = new.cName) > 10)
BEGIN
    UPDATE College SET cName = cName || '-done'
    WHERE cName = new.cName;
END;
```

When MIT's 11th application arrives, MIT is renamed MIT-done. (This will also trigger the cascade-update trigger from §15.3 — see **chaining** in §16.)

16. The Tricky Parts of Triggers

Once triggers can fire other triggers, complexity explodes. The lecture's part-2 demo exists specifically to illustrate these.

16.1 Self-triggering

A trigger on T whose action modifies T activates itself.

```
CREATE TRIGGER self
AFTER INSERT ON T1
FOR EACH ROW
BEGIN
    INSERT INTO T1 VALUES (new.val + 1);
END;
```

Insert 1 → trigger fires → inserts 2 → could fire again → could insert 3 → ...

SQLite default: A trigger may only fire **once per session** (to prevent infinite loops). Toggle with `PRAGMA recursive_triggers = ON;` to allow re-firing. Then **always add a termination condition**:

```
WHEN ((SELECT COUNT(*) FROM T1) < 10)
```

16.2 Cycles

R1 (on T1) inserts into T2 → R2 (on T2) inserts into T3 → R3 (on T3) inserts back into T1 → R1 fires again...

Same issue, same fix: a termination condition (or rely on the "fire once" default).

16.3 Conflicts — multiple triggers firing at once

Two triggers, same event, same data. Both updates affect the same rows. Result depends on **which trigger fires first**.

In the lecture's SQLite demo, **the trigger created last runs first** — but this is **implementation-specific**. Never rely on order: write triggers that produce the same result regardless of order, or use only one trigger per event.

16.4 Trigger chaining

One trigger's action causes a database change that activates *another* trigger. Healthy when intentional; nightmarish when it cascades unintentionally through the schema.

16.5 Nested invocation

A trigger's action has multiple statements; each statement may activate its own triggers, and those sub-activations happen **before** the next statement of the outer action runs. Result: nesting interleaves rows in unexpected orders.

16.6 Statement-level vs row-level — and SQLite's "immediate" quirk

This trips people up. Consider:

-- T1 has four rows: (1,1,1,1).

-- We bulk-insert four 2s:

```
INSERT INTO T1
```

```
SELECT value + 1 FROM T1;
```

-- Trigger inserts AVG(T1) into T2 after each insertion to T1.

Semantics	What appears in T2
SQL standard (trigger fires once at end-of-statement)	A single row: AVG = 1.5 (four 1s and four 2s)
SQLite/MySQL (immediate, per-row)	Four rows: progressively 1.2, 1.33, 1.43, 1.5 as 2s are inserted one by one

Lesson: When porting between DBMSs, this is one of the easiest places to get wrong. Always test the exact behaviour you expect.

16.7 Non-determinism

Some triggers can give different final states depending on the order of row processing — particularly row-level triggers that modify the rows they're processing. Avoid writing these unless you know exactly what you're doing.

17. Constraint Repair via Triggers

Ordinary constraints **reject** offending modifications. Triggers can **repair** instead.

Example: cap the high-school size silently rather than erroring:

```
CREATE TRIGGER cap_sizeHS
BEFORE UPDATE OF sizeHS ON Student
FOR EACH ROW
WHEN (new.sizeHS > 7000)
BEGIN
    SELECT RAISE(IGNORE);
END;
```

Or, do something useful — e.g., trim the value to a maximum rather than reject:

-- pseudo: depends on DBMS; SQLite needs INSTEAD OF style or a BEFORE that modifies new

The only built-in constraint mechanism that performs auto-repair is **referential integrity** with ON DELETE CASCADE / SET NULL / etc. (§7.5). Triggers are the general-purpose tool for auto-repair beyond that.

18. When to Use a Constraint vs. a Trigger

Use a constraint when...	Use a trigger when...
You can express the rule with NOT NULL, key, foreign key, or simple CHECK	You need cross-table conditions or aggregations
Rejection on violation is the correct behaviour	You need to auto-repair or take a custom action
You want the DBMS to use the info for optimisation (keys help the optimiser)	The DBMS doesn't support the constraint you need (e.g., MySQL CHECK, general assertions)
Standard SQL portability matters	Behaviour can vary by DBMS — confirm semantics carefully

Rule of thumb: prefer constraints when they suffice — they're simpler, more declarative, and more portable. Reach for triggers when you need expressive power or repair logic the constraint system can't deliver.

19. Summary

Constraints

- **Integrity constraints** restrict allowable database states beyond the schema.
- Six kinds, listed roughly in order of complexity: NOT NULL, PRIMARY KEY/UNIQUE, **referential integrity**, attribute-CHECK, tuple-CHECK, general assertion.
- Constraints are checked on every modification that could possibly violate them; violations roll back the modification.
- **DEFERRED checking** delays checks to end-of-transaction.
- **Referential integrity** is by far the most-used constraint in practice; remember the violation actions RESTRICT, SET NULL, SET DEFAULT, CASCADE on both DELETE and UPDATE.
- General assertions exist in the standard but **no DBMS implements them**.
- **MySQL silently ignores CHECK** — use PostgreSQL or SQLite for CHECK constraints.

Triggers

- **Triggers** are stored Event-Condition-Action rules: *when this event happens, if this condition is true, do that.*
- Two main uses: move monitoring logic into the DBMS, and **enforce/repair constraints** the constraint system can't express.
- Anatomy: timing (BEFORE/AFTER/INSTEAD OF) × event (INSERT/UPDATE/DELETE) × granularity (FOR EACH ROW vs statement) × references (OLD ROW, NEW ROW, OLD TABLE, NEW TABLE) × optional WHEN condition × action body.
- Implementations diverge a lot — **PostgreSQL closest to standard; SQLite is row-level + immediate; MySQL is row-level + immediate + one-trigger-per-event + no CHECK.**
- The hard parts: **self-triggering, cycles, chaining, nested invocations, conflicts between simultaneously firing triggers, and the row-level/immediate-activation quirk.** Use termination conditions and prefer triggers whose outcomes don't depend on firing order.

Quick syntax cheat sheet

-- Constraints in CREATE TABLE

col TYPE NOT NULL

col TYPE PRIMARY KEY

col TYPE UNIQUE

col TYPE CHECK (cond)

PRIMARY KEY (col1, col2)

UNIQUE (col1, col2)

CHECK (cond_over_tuple)

FOREIGN KEY (col) REFERENCES T(col)

ON DELETE {RESTRICT | CASCADE | SET NULL | SET DEFAULT}

ON UPDATE {RESTRICT | CASCADE | SET NULL | SET DEFAULT}

-- Trigger

CREATE TRIGGER name

{BEFORE | AFTER | INSTEAD OF} {INSERT | UPDATE [OF col] | DELETE} ON Table

[REFERENCING OLD ROW AS o NEW ROW AS n

OLD TABLE AS ot NEW TABLE AS nt]

[FOR EACH ROW]

[WHEN (condition)]

BEGIN

<SQL statements>

END;

*Constraints and triggers together let you encode meaningful, application-level rules **inside** the database itself rather than scattering them across application code. Used well, they make the database the guardian of its own integrity. Used poorly — especially with overlapping triggers — they create some of the hardest debugging puzzles in all of software.*